Kernel Linux

Description	Architettura sistema android

Si tratta di un kernel completo, con tutte le primitive UNIX.
Processi, gestione della memoria, IPC, thread

- Filesystem, utenti, diritti
- Librerie, shell, comandi
- Driver (sotto forma di moduli) per vari device
  - Tipicamente, quelli presenti in ogni particolare dispositivo
  - SD card, reti, telefonia, sensori, ecc.

Applicazioni native

È possibile scrivere applicazioni che chiamano direttamente il kernel

- Direttamente (via syscall) o via librerie (es. stdio.h)
- Il codice deve essere compilato per il particolare processore in uso su un certo telefonino
  - In genere, ARM – ma non forzosamente
- Deve poi essere “impacchettato” in un formato specificato per la distribuzione/installazione
  - Non troppo diverso dai vari .rpm, .deb, e simili

Dalvik & ART

La stragrande maggioranza delle applicazioni gira su una macchina virtuale: Dalvik

Analoga alla JVM con importanti differenze:

- Basata su registri (non su stack)

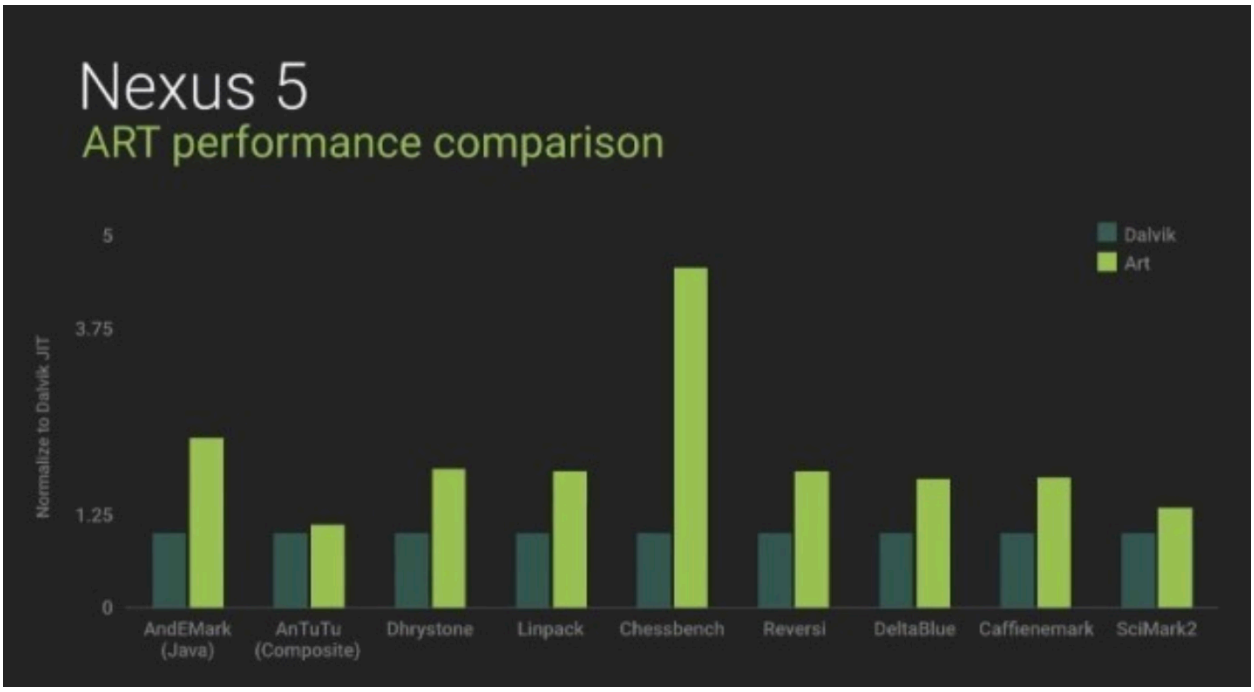
- Set di istruzioni ottimizzato per risparmiare memoria e aumentare la velocità di esecuzione
- Formato dei file eseguibili ottimizzato per risparmiare memoria
- Eseguibile da più processi con una sola istanza
    - Tutto codice rientrante – sharing del codice di Dalvik via mmap()

Dalvik e ART (Android Runtime) sono due ambienti di esecuzione utilizzati nel sistema operativo Android per eseguire le applicazioni Android. Ecco alcune differenze tra Dalvik e ART:

1. **Interpretazione vs Compilazione AOT (Ahead-of-Time):**
    - **Dalvik:** Utilizza un'approccio di interpretazione just-in-time (JIT), che traduce il bytecode delle applicazioni in istruzioni macchina mentre l'app viene eseguita.
    - **ART:** Utilizza un approccio di compilazione ahead-of-time (AOT), che traduce il bytecode delle applicazioni in istruzioni nativamente eseguibili durante l'installazione dell'applicazione.
2. **Velocità di Esecuzione:**
    - **Dalvik:** Poiché utilizza l'interpretazione JIT, può richiedere più tempo per avviare un'applicazione e può comportare una leggera penalità sulle prestazioni durante l'esecuzione iniziale.
    - **ART:** La compilazione AOT di ART migliora le prestazioni durante l'esecuzione dell'applicazione, poiché il codice è già convertito in istruzioni nativamente eseguibili, riducendo il ritardo durante l'avvio e migliorando la responsività complessiva del sistema.
3. **Ottimizzazione della Batteria:**
    - **Dalvik:** Potrebbe richiedere più risorse CPU durante l'esecuzione a causa dell'interpretazione JIT, potenzialmente influenzando la durata della batteria.

- **ART:** La compilazione AOT può contribuire a migliorare l'efficienza energetica, poiché il codice è già compilato durante l'installazione e non richiede la stessa quantità di elaborazione in tempo reale durante l'esecuzione dell'app.

4. **Garbage Collection:**
   - **Dalvik:** Utilizza un modello di garbage collection che può portare a picchi di utilizzo della CPU durante la pulizia della memoria non utilizzata.
   - **ART:** Introduce miglioramenti nella gestione della memoria, inclusa una garbage collection più efficiente che può ridurre i ritardi associati alla pulizia della memoria.
5. **Spazio di Archiviazione:**
   - **Dalvik:** Richiede generalmente meno spazio di archiviazione poiché il bytecode è più compatto.
   - **ART:** La compilazione AOT può richiedere più spazio di archiviazione, poiché il codice viene tradotto in istruzioni native, occupando quindi più spazio su disco.
6. **Supporto per le Nuove Funzionalità:**
   - **Dalvik:** Potrebbe essere limitato nel supporto delle ultime funzionalità e ottimizzazioni del linguaggio Java.
   - **ART:** È progettato per fornire un migliore supporto per le ultime caratteristiche del linguaggio Java e può sfruttare ottimizzazioni avanzate.



Dalvik

1️⃣ ***javac:*** .java → .class
dx: .class → .dex

2️⃣ dexopt: .dex → .odex

3️⃣ libdvm.so:
.odex → run
– Interpretato + JIT

ART

1️⃣ ***javac:*** .java → .class
dx: .class → .dex

2️⃣ dex2oat: .dex → ELF

3️⃣ libart.so: ELF → run
– Esecuzione nativa con un po' di runtime

compile-time

install-time

run-time

Ogni App viene eseguita dal kernel Linux

- In un processo separato
  - Che esegue Dalvik che esegue il bytecode dell'app
  - Controllo dei permessi di accesso alle risorse logiche fatto dalla VM (i permessi sono concessi dall'utente-umano)
- Con uno user ID distinto
  - Tutti i file creati dall'applicazione appartengono al “suo” user ID, altre applicazioni non possono accedere alla “sua” directory, né leggere i “suoi” file
  - È possibile che applicazioni amiche condividano processo e user ID, occorre però che siano firmate dallo stesso autore
  - Controllo dei diritti di accesso alle risorse fisiche fatto dal kernel (i diritti non sono controllati dall'utente-umano)